

Lesson2

认识对象：封装数据为类

主讲老师：高祥



2024/9/12



1 北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

本节主要内容：

- 对象与类的概念
- 类的设计与定义
 - 成员变量定义
 - 成员方法定义
 - 构造函数定义
- 类之间的关系

- 类：在软件中，类，就是一个模板，它定义了一类事物的状态和行为。
- 类是对现实世界的抽象，因此类是一种抽象的复合数据类型。

- 对象是现实世界中的一个实体，其特征是：
 1. 对象的标示（名称）唯一；
 2. 对象的状态（属性、数据区）
 3. 对象的行为（方法、功能）

类与对象的关系

- 类和对象之间是抽象和具体的关系。类是创建对象的模板，对象是类的具体实例。
- 类（class）是总称，对象是个体，因此对象（object）也叫实例（instance）。

面向对象的思维：（overview）

- 第一步：明确问题域，分析这个问题里面有哪些类和对象
- 第二步：分析这些类和对象应该具有哪些属性和方法。
- 第三步：分析类和类之间具体有什么关系。

分析的过程就是抽象的过程



1 个工具

① 抽象 (Abstract)

■ 摒弃细节，提取共性

2 个概念

① 对象 (Object)

- 客观存在的实体

② 类 (Class)

- 具有相同性质的对象的抽象体

3 个特性

- ① 封装 (Encapsulation)
- ② 继承 (Inheritance)
- ③ 多态 (Polymorphism)

4 个步骤

① 分析 (Analysis)

- 找出系统中的对象，抽象出类，确定它们所属的不同主题，分析它们之间的关系

② 设计 (Design)

- 对每个类应该封装的数据及其操作进行详细设计和描述

③ 实现 (Implementation)

- 采用某种编程语言编码 (Coding) 实现各个类

④ 测试 (Test)

- 由类创建对象，验证其功能和性能是否满足需求

案例分析（1）：

- 小王本来体重70Kg，经过减肥，体重降到45Kg，试从这个问题领域中识别对象、类、属性、行为、状态，和状态变化。
- 答案？

- 对象：小王
- 类：人
- 属性：体重
- 行为：减肥
- 状态：减肥前状态：小王体重70Kg；减肥后状态：小王体重45Kg
- 状态变化：减肥行为使得小王的体重发生了变化

案例分析（2）：

- 假如某公司需要买组装电脑：
 - ① 在网上查询具体每一个硬件的参数和报价；
 - ② 实体店询价；
 - ③ 询价结束后，根据具体的结果分析出自己比较满意的某品牌报价；
 - ④ 到某品牌店里进行组装，组装时需要进行现场监督。



案例分析 (2)

- 对象：公司、采购、组装工程师、硬件、硬件供应商.....
- 类：公司类、员工类、硬件类、供应商类、测试类等
- 属性：硬件类---名称，价格，硬件参数，供应商
- 行为：员工---询价、报价、组装

本节主要内容：

- 对象与类的概念
- 类的设计与定义
 - 成员变量定义
 - 构造函数定义
 - 成员方法定义
- 类之间的关系



Java程序的基本结构

- 一个基本Java程序的三大件

- 1、包的声明；//指定类所在的位置

- 包命名常采用顶级域名作为前缀，例如com, net, org, edu, gov, io等，随后紧跟公司/组织/个人名称以及功能模块名称（cn.edu.buaa.oop）

- 2、类的导入；//用到其它位置中的类

- 3、类的定义；//核心部分

源文件基本语法

- Java 源文件基本语法

- {<包声明>}

- {<导入声明>}

- {<类声明>}

- 示例：HelloWorld.java 文件：

```
1  package cn.edu.buaa.oop.lesson_1;
2
3  import cn.edu.buaa.oop.lesson_2.Student;
4
5  public class HelloWorld {
    Run | Debug
6      public static void main(String[ ] args) {
7          System.out.println(x: "Hello world!");
8          Student stu1 = new Student();
9      }
```


Student.java (一个类放在一个源文件中)

```
1  package cn.edu.buaa.oop.lesson_2;
2
3  public class Student {
4      private String studentId;
5      private String studentName;
6      private int studentAge;
7      private int OopGrade;
8
9      public Student(){
10
11  }
12 }
```

定义一个类的步骤

- 一. 定义类名
- 二. 编写类的属性
- 三. 编写类的方法

数据封装

类的定义（[]代表可有可无）

- 类的定义格式如下：

```
[类修饰符] class 类名 extends 基类  
implements 接口列表  
{  
    [数据成员定义]  
    [成员方法定义]  
}
```

关键字class表示类定义的开始

类名要符合标识符的命名规范

修饰符分为
访问控制符和
类型说明符

类的定义说明

- 一. 关键字`class`表示类定义的开始
- 二. 类名要符合标识符的命名规范
 - 1. 首字母大写
 - 2. 一个类名包含两个以上名词，建议使用驼峰命名 (Camel-Case)法 (**S**tudent**T**est)
- 三. 修饰符 (`modifier`) 分为访问控制符 (`public`) 和类型说明符 (`abstract`)

- 类的访问控制符有两个，一个是`public`，即公共类，另一个就是默认，即没有访问控制符。
 - ① 一个类被定义为公共类，就表示它能够被其它所有的类访问和引用。
 - ② 在一个Java源程序中只能有一个`public`类，这个类一般含有`main`方法。
 - ③ 不用`public`定义的类，其只能被同一个包中定义的类访问和引用。
 - ④ 在一个JAVA程序中可以定义多个这样的类。

类的设计原则

- 一. 取有意义的名字。
- 二. 尽量将数据设计为私有属性。
- 三. 尽量对变量进行初始化。
- 四. 类的功能尽量单一（原子、细粒度）。

Student.java（多个类放在一个源文件中）

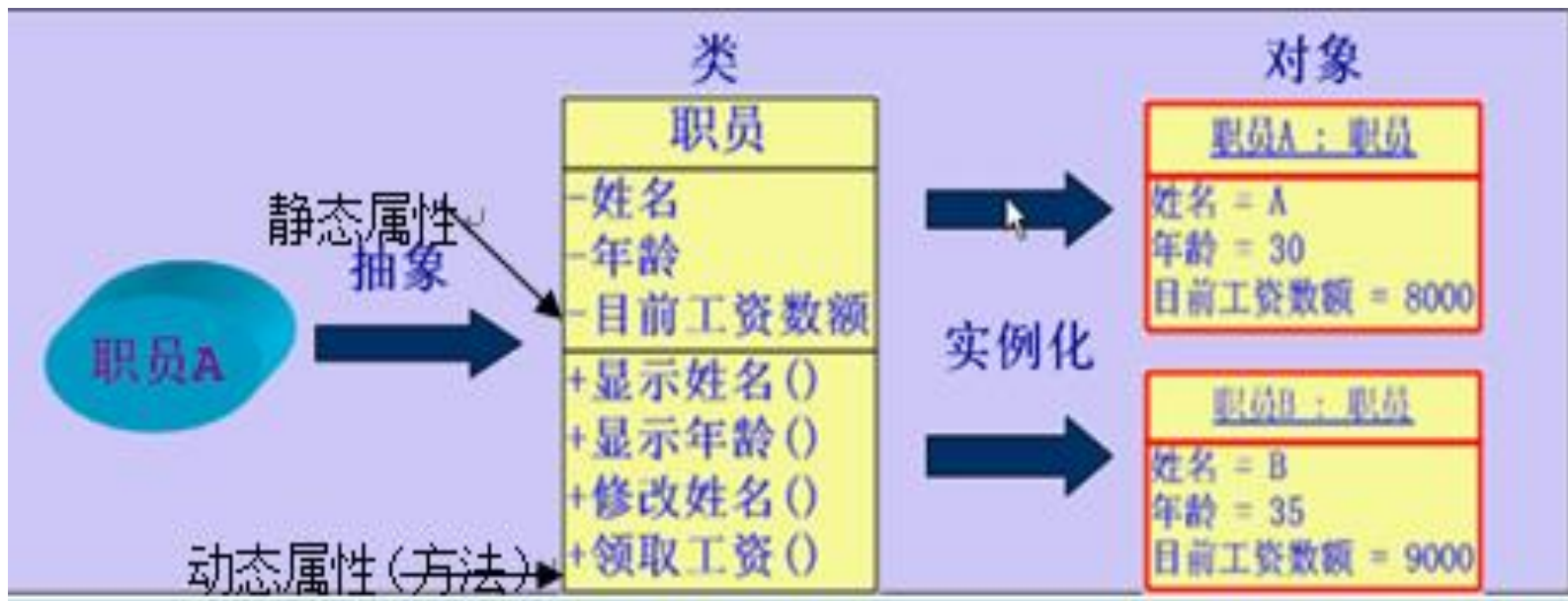
```
1  package cn.edu.buaa.oop.lesson_2;
2
3  public class Student {
4      private String studentId;
5      private String studentName;
6      private int studentAge;
7      private int OopGrade;
8
9      public Student(){
10
11  }
12 }
```

```
13
14 class StudentTest{
15
16 }
```

Java 允许在一个 Java 源文件中编写多个类，但其中的多个类至多只能有一个类使用 public 修饰。

类的定义-类属性分析

- 有两个方面，一方面是它的静态属性(状态)，另一方面是它的动态属性(行为)。**反映到JAVA里面的类怎么包装它呢？一方面成员变量，另一方面是方法。**
- 例如：职员这个类该怎么抽象出来？
 - 也是从两个方面，一方面是它的静态属性，另

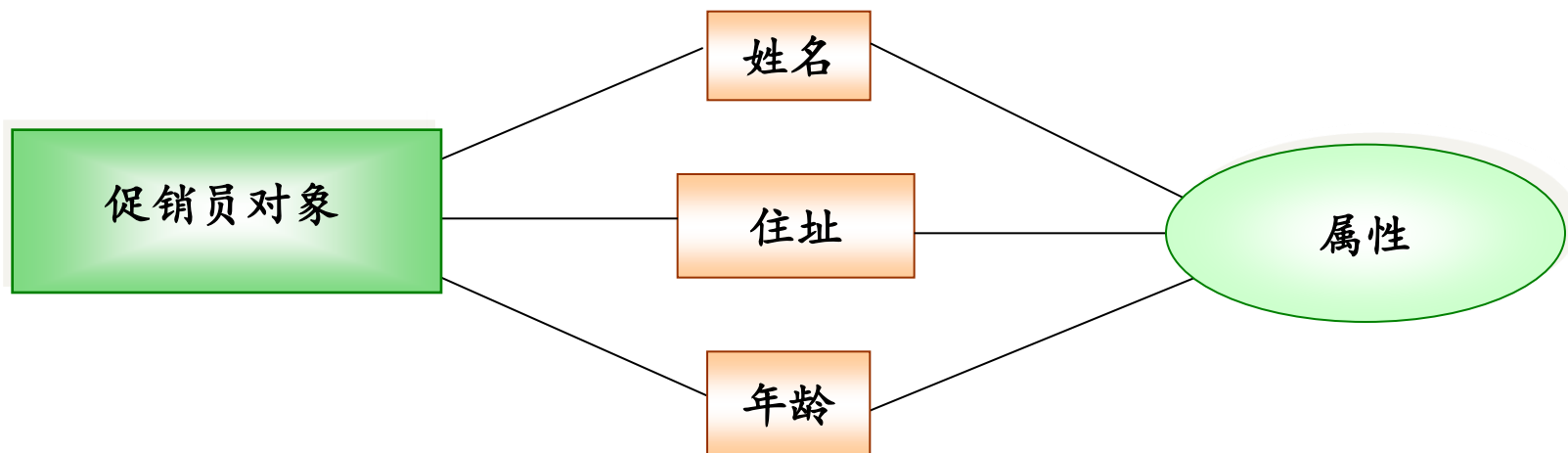


本节主要内容：

- 对象与类的概念
- 类的设计与定义
 - 成员变量定义
 - 构造函数定义
 - 成员方法定义
- 类之间的关系

成员变量

- 事物的特性,在类中表示为变量
- 每个对象的每个属性都拥有其特有的值
- 属性名称由类的所有实例共享



“在类中表示对象或实体拥有的特性时称为属性”

成员变量的定义（[]代表可有可无）

- 成员变量的定义格式：
 - [修饰符] 变量的数据类型 变量名 [=初始值]
- 修饰符主要有四种，分别是：
 - ① public、
 - ② private、
 - ③ protected、
 - ④ final、
 - ⑤ this(大多数可以省略，实例变量)、
 - ⑥ static(类变量)、
 - ⑦ 默认

访问级别

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	X
no modifier	Y	Y	X	X
private	Y	X	X	X

From: <https://blog.csdn.net/w372426096/article/details/78012551>



- 类的成员变量可以是基本类型或数组，也可以是类的对象。
- 类的成员方法用于处理该类的数据，是用户与对象之间或对象之间的交互接口。

成员变量类型

- 类的设计是细粒度的
 - 例如：公司员工拥有地址，包括公司地址和家庭地址，一般我们会单独编写地址类（Address）

```
class Address{  
  
}  
  
class Employee{  
    Address homeAddress;  
    Address ComAddress;  
    .....  
}
```

- 从编程需求出发，不需要的不考虑，所有的属性和方法，都是在编程中所需要用到的。

- 首字母小写
- 一个变量名包含两个以上名词，从第二词开始建议使用驼峰命名(Camel-Case)法（studentName）

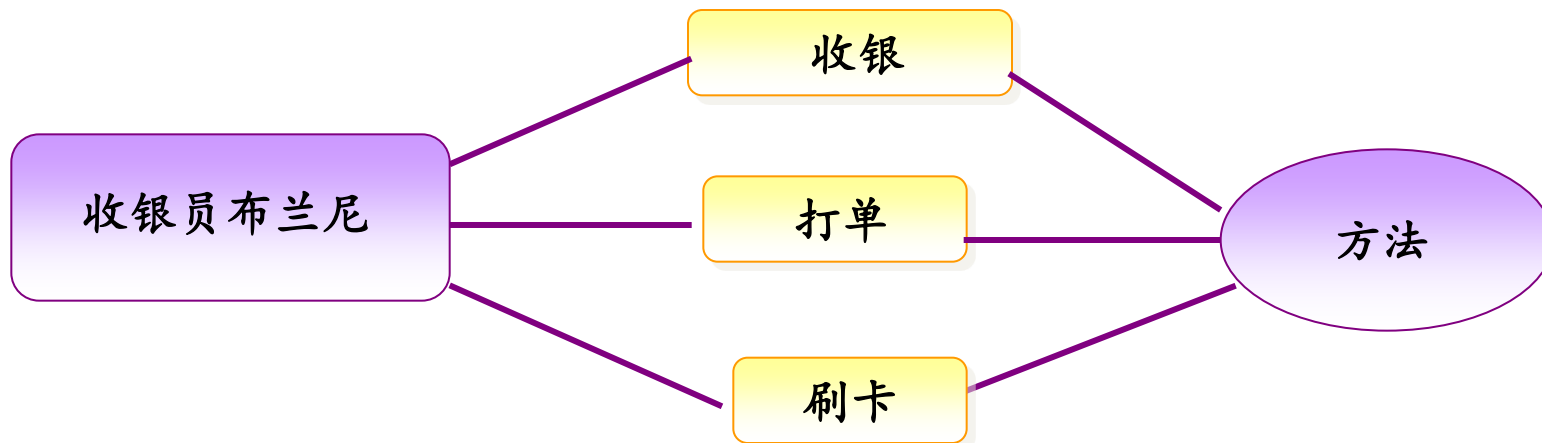
成员变量的设计原则

- 一. 取有意义的名字。
- 二. 尽量将数据设计为私有属性。
- 三. 尽量对变量进行初始化。

本节主要内容：

- 对象与类的概念
- 类的设计与定义
 - 成员变量定义
 - 成员方法定义
 - 构造函数定义
- 类之间的关系

- 方法：对象执行的操作



方法

操作的实际实现

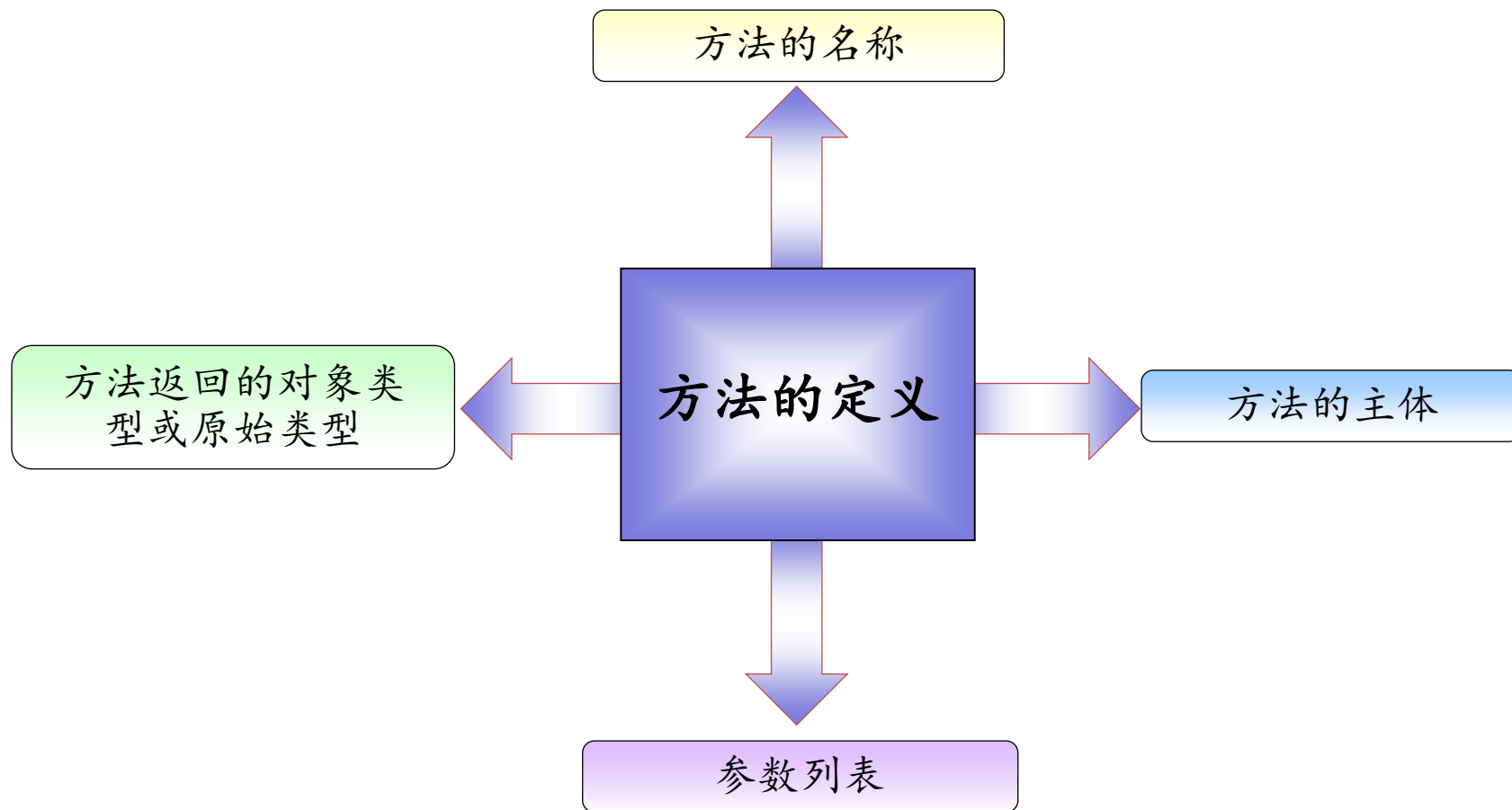
方法指定操作对象
数据的方式

如何执行所请求的
操作的规范

在得到操作请求时
指定如何做的算法

“对象执行的操作称为方法。”

类中的方法



成员方法的定义（[]代表可有可无）

- 方法的定义格式：

```
[修饰符] 返回值类型 方法名([形参说明])  
[throws 例    例外名1, 例外名2...]  
{  
    局部变量声明;  
    执行语句组;  
}
```

- 首字母小写
- 一个方法名包含两个以上名词，从第二词开始建议使用驼峰命名(Camel-Case)法（getStudentName）
- 方法名第一个单词为动词

成员方法的定义说明

- 一. 常用的修饰符为public、private、protected、static、final等
- 二. 返回值类型：方法一般需要有一个返回值表示执行结果，也可以无返回值（用void表示）。
返回值类型可以是Java类型系统中的所有类型。
- 三. 有返回值的方法使用return语句将值返回给调用者。

局部变量和成员变量区别

- 区别一：定义的位置不同
 - 定义在类中的变量是成员变量
 - 定义在方法中或者{}语句里面的变量是局部变量
- 区别二：在内存中的位置不同
 - 成员变量存储在堆内存的对象中
 - 局部变量存储在栈内存的方法中

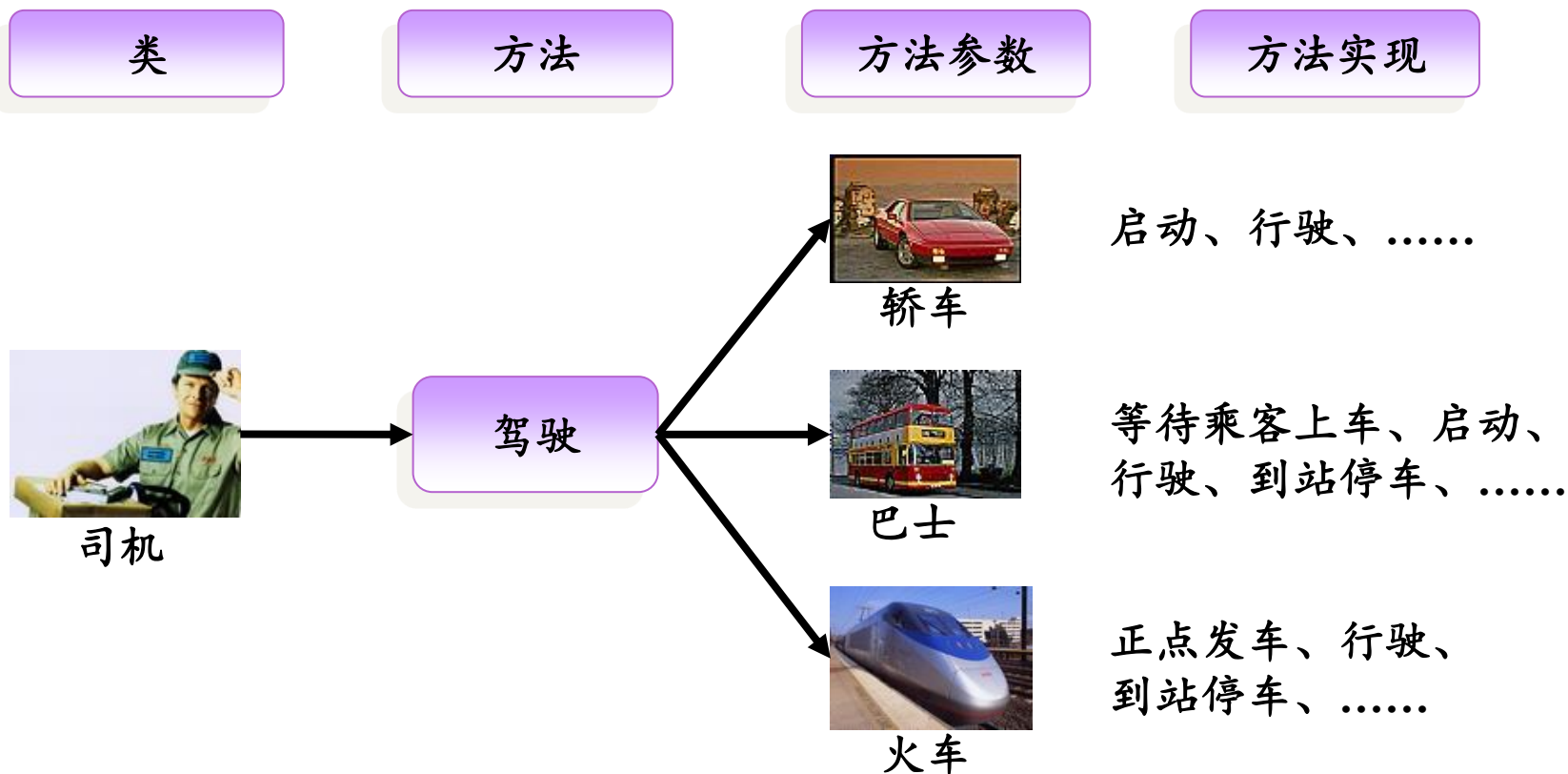
局部变量和成员变量区别

- 区别三：声明周期不同
 - 成员变量随着对象的出现而出现在堆中，随着对象的消失而从堆中消失
 - 局部变量随着方法的运行而出现在栈中，随着方法的弹栈而消失
- 区别四：初始化不同
 - 成员变量因为在堆内存中，所有成员变量具有默认的初始化值
 - 局部变量没有默认的初始化值，必须手动的给其赋值才可以使用。

方法的重载

- 方法的重载是指一个类中可以定义有相同的名字，但参数不同的多个方法，调用时会根据不同的参数表选择对应的方法。
- 调用的方法实现由参数个数和参数类型确定

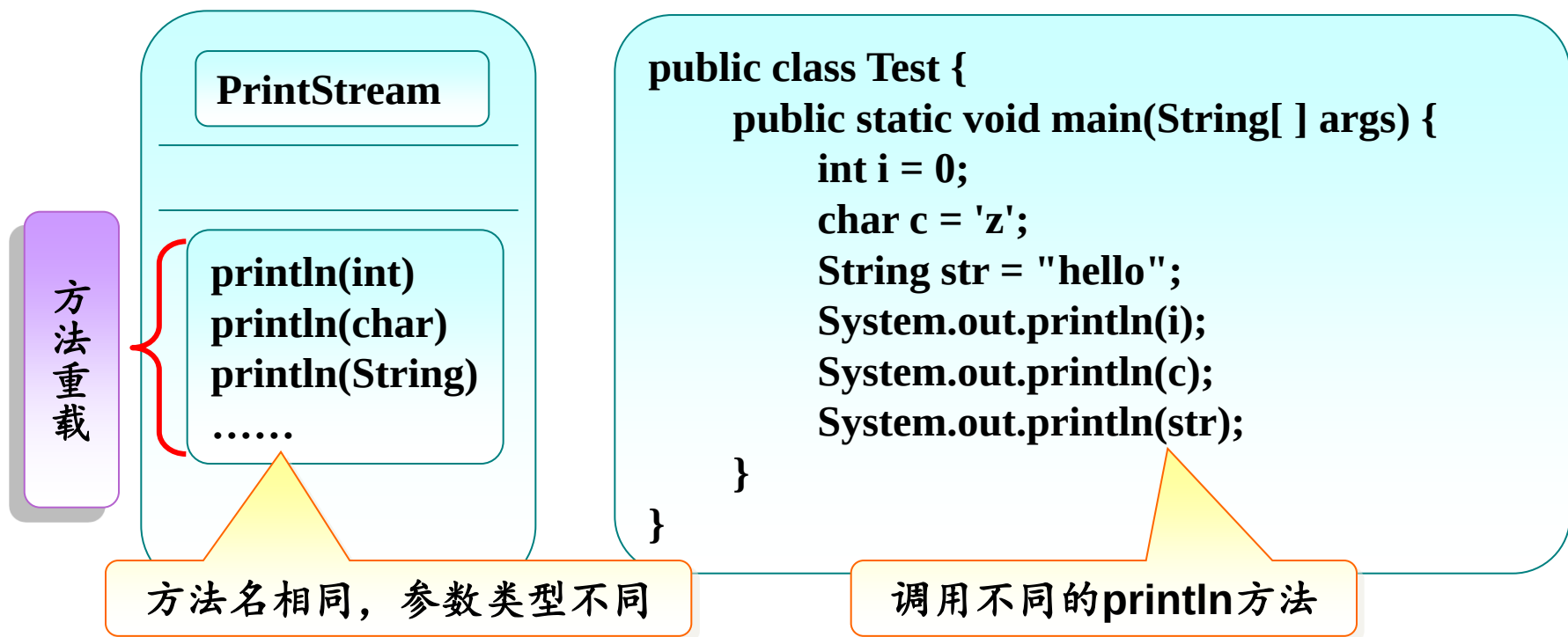
生活中的方法重载



如果用代码实现，我们需要三个方法，这些方法的方法名称相同，参数类型不同

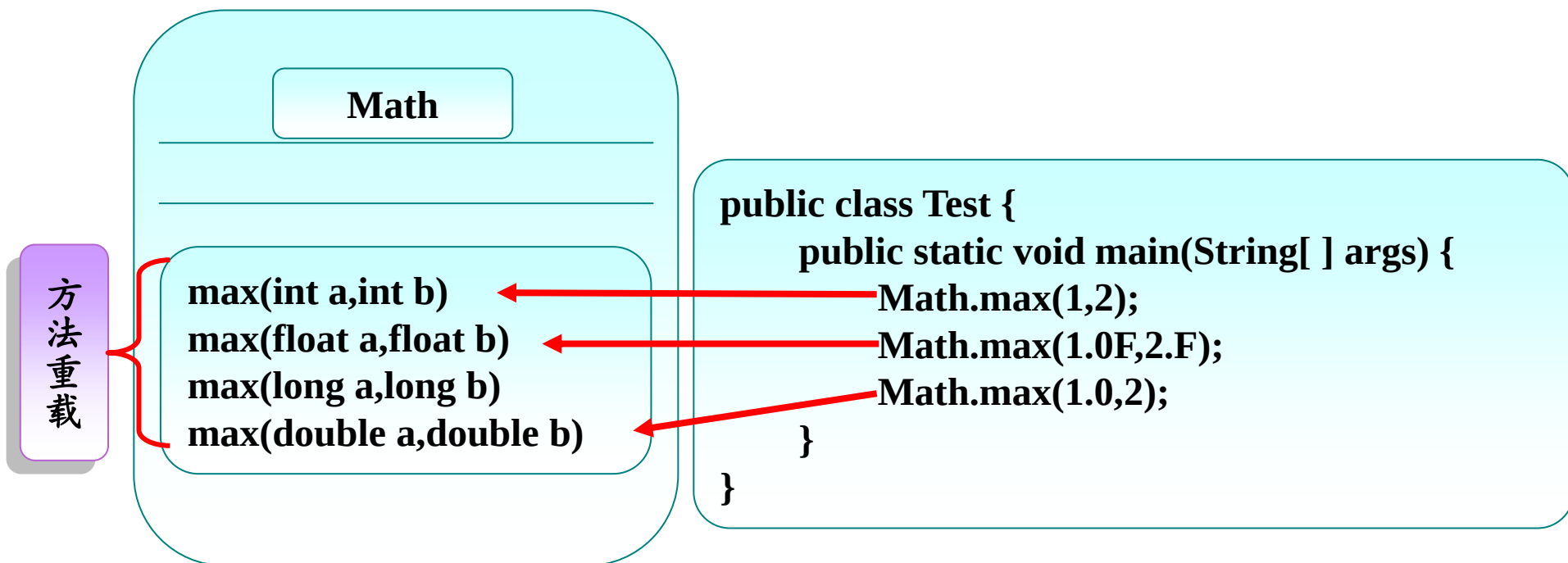
方法重载的代码示例

- java.io.PrintStream类的println方法能够打印数据并换行，根据数据类型的不同，有多种实现方式



方法重载的代码示例

- `java.lang.Math`类的`max()`方法能够从两个数字中取出最大值，它有多种实现方式
- 运行时，Java虚拟机先判断给定参数的类型，然后决定到底执行哪个`max()`方法

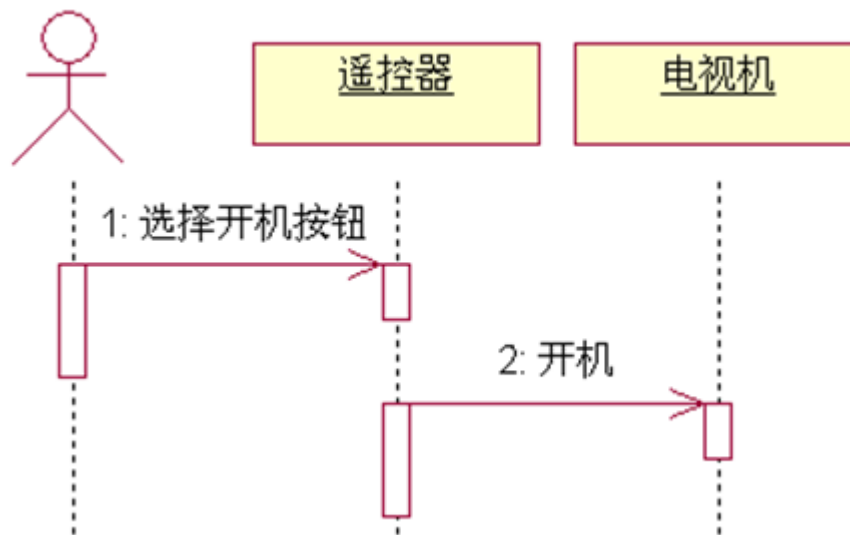


toString() 方法 （特殊点强调）

- 在java中，所有对象都有默认的toString() 这个方法
- 创建类时没有定义toString() 方法，输出对象时会输出对象的哈希码值（对象的内存地址）
- 它通常只是为了方便输出，比如
System.out.println(xx)，（xx是对象），括号里面的”xx”如果不是String类型的话，就自动调用xx的toString() 方法
- 它只是sun公司（Oracle）开发java时为了方便所有类的字符串操作而特意加入的一个方法

如何理解“消息”

- 对象提供的服务是由对象的方法来实现的，因此发送消息实际上也就是调用一个对象的方法。
- 例如：遥控器向电视机发送“开机”消息，意味着遥控器对象调用电视机对象的开机方法
 - television.open(); //遥控器对象调用电视机对象的开机方法**



本节主要内容：

- 对象与类的概念
- 类的设计与定义
 - 成员变量定义
 - 成员方法定义
 - 构造函数定义
- 类之间的关系

创建对象

- 一. 必须使用new关键字创建一个对象
- 二. 使用对象属性（对象名.成员变量）
- 三. 使用对象方法（对象名.方法名）
- 四. 同一个类的每个对象有不同的成员变量的存储空间
- 五. 同一个类的每个对象共享该类的方法

构造函数(Constructor)

- 一. 构造方法(constructor)是一类特殊的成员方法。
- 二. 从功能上讲, 它使用new关键字用来对新创建的对象进行初始化的。
- 三. 从形式上来讲, 它有以下特点:
 - ① 它与类同名;
 - ② 它没有任何返回值;
 - ③ 除了上述两点外, 在语法结构上与一般的方法相同。

- Java语言中，每个类都至少有一个构造方法；
- 构造方法有两大类，构造方法可以重载，并且通常是重载的：
 - 不带参数构造方法（默认构造方法）
 - 带参数构造方法

为什么需要构造方法

```
public class Teacher {  
    private String name;      // 姓名  
    private int age;         // 年龄  
    private String education; // 学历  
    private String position;  // 职位  
  
    public String introduction() {  
        return "大家好！我是" + name + "，我今年" + age + "岁，学历“  
            + education + "，目前职位是" + position;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String myName) {  
        name = myName;  
    }  
    // 以下是其他属性的setter、getter方法，此处省略  
}
```

为什么需要构造方法

```
public class Teacher4Test {  
    public static void main(String[ ] args) {  
        Teacher teacher = new Teacher();  
        teacher.setName("Mary");  
        teacher.setAge(23);  
        teacher.setEducation("本科");  
        System.out.println(teacher.introduction());  
    }  
}
```

Teacher4中有太多的属性及对应的**setter**方法，在初始化时，**很容易就忘记了，造成代码冗余**，有没有可能简化对象初始化的代码？

大家好！我是**Mary**，我今年**23**岁，学历**本科**，目前职位是**null**

要简化对象初始化的代码，可以通过构造方法来解决

不带参数构造方法（默认构造方法）

```
public class Teacher {  
    private String name; // 教员姓名  
    // 构造方法  
    public Teacher() {  
        name = "Mary";  
    }  
}
```

```
public class TeacherTest {  
    public static void main(String[ ] args) {  
        Teacher5 teacher = new Teacher();  
    }  
}
```

带参数的构造方法

- 通过带参数的构造方法，显式地为实例变量赋予初始值

带参数的构造方法

```
public class Teacher {  
    private String name;    // 教员姓名  
    private int age;        // 年龄  
    private String education; // 学历  
    private String position; // 职位  
    // 带参数的构造方法  
    public Teacher(String pName,int pAge,String pEducation,String pPosition) {  
        name = pName;  
        age = pAge; // 可以增加对age等属性的存取限制条件  
        education = pEducation;  
        position = pPosition;  
    }  
    public String introduction() {  
        return "大家好！我是" + name + "，我今年" + age + "岁，学历" +  
            education + "，目前职位是"+position;  
    }  
}
```


带参数的构造方法

- 通过调用带参数的构造方法，简化对象初始化的代码

```
public class Teacher6Test {  
    public static void main(String[] args) {  
        Teacher6 teacher = new Teacher("Mary",  
                                     23, "本科", "咨询师");  
        System.out.println(teacher);  
    }  
}
```

创建对象时，一并完成了对象成员的初始化工作

大家好！我是Mary,我今年23岁，学历本科，目前职位是咨询师

构造函数(Constructor)

- 构造函数实例化类对象的格式
 - 类名 对象名=new 构造函数 (实际参数)
 - New关键字的作用
 - ① 为对象分配内存空间。
 - ② 引起对象构造方法的调用。
 - ③ 为对象返回一个引用。

不带参数构造方法（默认构造方法）

- Java语言中，每个类都至少有一个构造方法；
- 如果类的定义者没有显式的定义任何构造方法，系统将自动提供一个默认的构造方法：
 - 默认构造方法没有参数
 - 默认构造方法没有方法体
 - 默认的构造方法：`Animal() {}`
- 所以：不编写构造方法就能用`new Xxx()`创建类的实例。

不带参数构造方法（默认构造方法）

- 无参构造函数创建对象时，成员变量的值被赋予了数据类型的隐含初值。
 - 例如：整型是 0，实型是 0.0，复合类型是 null。

变量类型	默认值	变量类型	默认值
byte	0	short	0
int	0	long	0L
float	0.0f	double	0.0d
char	'\u0000'	boolean	false
引用类型	null		

不带参数构造方法（默认构造方法）

- 在Java里，如果一个类没有明显的表明哪一个类是它的父类，**Object**类就是它的父类。
- 如果类中没有定义构造函数，编译器会自动创建一个默认的不带参数的构造函数。
- 如果程序员为类定义了构造函数，Java就不会为该创建默认的不带参数的构造函数。
- 注意：如果类中提供的构造函数都不是无参数构造函数，却企图调用无参数构造函数初始化此类的对象，编译时会产生语法错误

- 通过调用不同的构造方法来表达对象的多种初始化行为
- 例如：
 - 默认情况下，教师来自北京中心，初始化时，只需提供教员姓名
 - 有时，需要提供所在中心名称及教员姓名

构造方法重载是方法重载的典型示例

构造方法重载

```
public class Teacher {  
    private String name; // 教员姓名  
    private String school = "北航"; // 所在中心  
  
    public Teacher(String name) {  
        this.name = name; // 设定教员姓名  
    }  
    public Teacher(String name, String school) {  
        this.name = name; // 设定教员姓名  
        this.school = school; // 设定教员的所在中心  
    }  
    public String introduction() {  
        return "大家好! 我是" + school + "的" + name;  
    }  
}
```

通过调用不同的构造方法来表达对象的多种初始化行为

```
public class TeacherTest {  
    public static void main(String[ ] args) {  
        Teacher teacher1 = new Teacher("Mary");  
        System.out.println(teacher1.introduction());  
  
        Teacher teacher2 = new ATeacher("Jack", "北航");  
        System.out.println(teacher2.introduction());  
    }  
}
```

大家好！我是北京中心的Mary
大家好！我是天津中心的Jack

- 给定如下Java代码，请指出代码中的错误，并解释原因。

回答问题

```
public class Sample {  
    private int x;  
    ✓ public Sample() {  
        x = 1;  
    }  
    ✓ public Sample(int i) {  
        x = i;  
    }  
    ✓ public int Sample(int i) {  
        x = i;  
        return x++;  
    }  
    ✓ private Sample(int i, String s){}  
    ✓ public Sample(String s,int i){}  
    ✗ private Sampla(int i){  
        x=i++;  
    }  
    ✓ private void Sampla(int i){  
        x=i++;  
    }  
}
```

无参构造方法

带参构造方法

不是构造方法

带参构造方法

带参构造方法

名称与类名不相同

不是构造方法

本节主要内容：

- 对象与类的概念
- 类的设计与定义
 - 成员变量定义
 - 构造函数定义
 - 成员方法定义
- 类之间的关系

- UML 的主要目的是：
- 支持关于所开发系统的分析和设计的沟通
- 支持从“现实世界”的问题域转向机器中的解决方案域
 - 对于同一系统，有两种视角：
 - 一个视角有图表
 - 源代码是另一个视角

The Unified Modeling Language (UML)

- UML 是一种用于描述软件设计的图形符号
它有关于如何绘制以下模型的规则：
 - 类
 - 类之间的关联
 - 对象之间的消息传递
- UML 已成为事实上的行业标准
 - 虽然不是官方标准，但几乎所有人都在使用它
- 就像一张蓝图，展示分析、设计和实现过程中发生的事情

- 表达要实现的类定义
 - 列出每个类的名称、属性和方法
 - 展示类之间的关系
- UML 允许对一个类的属性和方法提供不同级别的细节
 - 可以只是一个矩形中的类名
 - 也可以类的详细信息

Software Specification (Class Name)
attribute attribute : type attribute : type = initial value ...
method1() method2(parameter : Type) : return type <i>abstractMethod()</i> +publicMethod() -privateMethod() #protectedMethod() ...

◆ The Relationships in UML

- 1) 依赖
- 2) 关联
- 3) 组合
- 4) 泛化
- 5) 实现

类之间的关系

- 类之间分为以下五种关系：
 - 关联：类A与类B的实例之间存在特定的对应关系。
 - 依赖：类A访问类B提供的服务。
 - 聚集：类A为整体类，类B为局部类，类A的对象由类B的对象组合而成。
 - 泛化：类A继承类B。
 - 实现：类A实现了B接口。

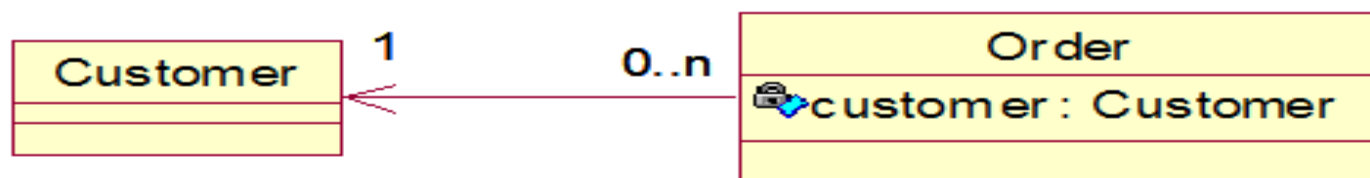


关联关系 (Association)

- 关联指的是类之间的特定对应关系，在UML中用带实线的箭头表示。
- 按照类之间的数量对比，关联可分为以下三种：
 - **一对一关联**：例如假定一个家庭教师只教一个学生，一个学生只有一个家庭教师，那么家庭教师和学生之间是一对一关联。
 - **一对多关联**：例如假定一个足球队员只能加入一个球队，一个球队可以包含多个队员，那么球队和队员之间是一对多关联。
 - **多对多关联**：例如假定一个足球队员可以加入多个球队，一个球队可以包含多个队员，那么球队和队员之间是多对多关联。



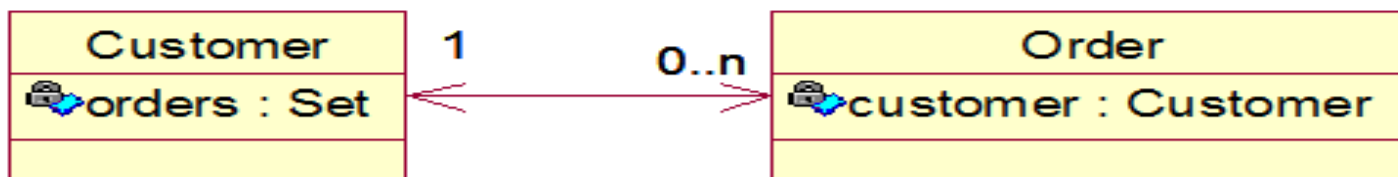
客户和订单的关联关系



从 Order 到 Customer 的多对一单向关联



从 Customer 到 Order 的一对多单向关联



Customer与Order的双向关联

Customer 类关联 Order 类

```
public class Customer {  
    ...  
    /** 所有与Customer对象关联的Order对象 */  
    private Set<Order> orders=new HashSet<Order>();  
    public Set getOrders() {  
        return orders;  
    }  
  
    public void setOrders(Set orders) {  
        this.orders = orders;  
    }  
}
```



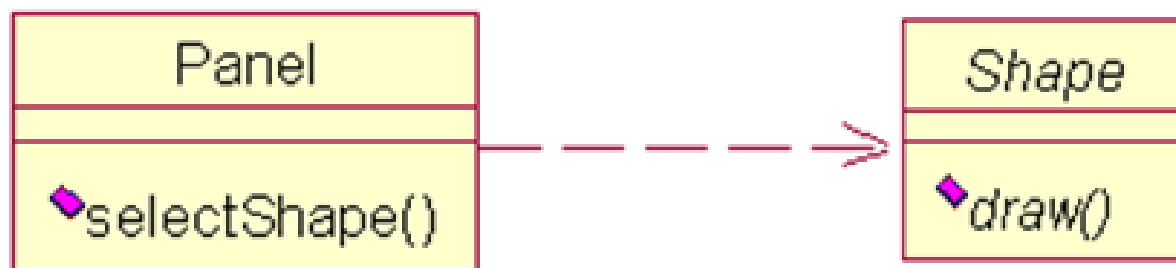
Order 类关联Customer 类

```
public class Order {  
    ...  
    /** 与Order对象关联的Customer对象 */  
    private Customer customer;  
    public Customer getCustomer() {  
        return this.customer;  
    }  
    public void setCustomer(Customer customer) {  
        this.customer = customer;  
    }  
}
```



依赖关系 (Dependency)

- 依赖指的是类之间的调用关系，在UML中用带虚线的箭头表示。如果类A访问类B的属性或方法，或者类A负责实例化类B，那么可以说类A依赖类B。
 - 例如：把大象装进冰箱
 - 例如：在面板中绘图



案例分析：

- 问题域：
 - 狗逮耗子
 - Mouse 类
 - Dog 类

代码分析 (Mouse类)

```
package com.buaa.demo;
public class Mouse {
    private String name;
    private double height;
    private double weight;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public double getHeight() {
        return height;
    }
    public void setHeight(double height) {
        this.height = height;
    }
    public void scream() {
        System.out.println("我被狗咬到了，好痛啊！");
    }
}
```

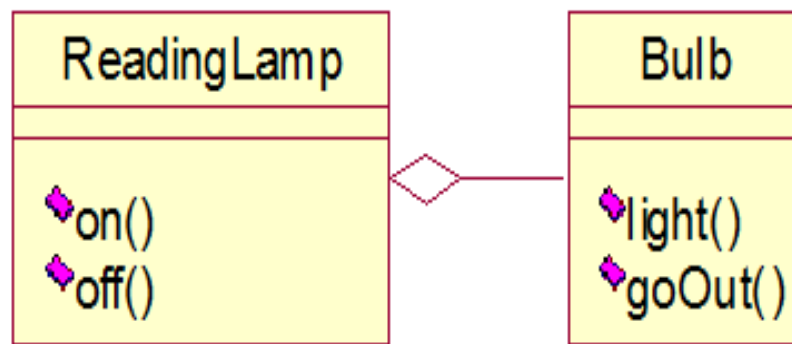

代码分析 (Dog类)

```
package com.buaa.demo;
public class Dog {
    private int furcolor; //定义属性: 毛的颜色
    private float height; //定义属性: 狗的高度
    private float weight; //定义属性: 狗的体重
    public Dog(){
    public Dog(int furcolor, float height, float weight) {
        super();
        this.furcolor = furcolor;
        this.height = height;
        this.weight = weight;
    }
    public void CatchMouse(Mouse m){
        m.scream();
    }
    public static void main(String[] args) {
        Dog d = new Dog(); //首先用new关键字创建一只狗
        Mouse m=new Mouse(); //造出一只老鼠。
        d.CatchMouse(m); //然后用这只狗去抓老鼠, 调用CatchMouse()
    }
}
```

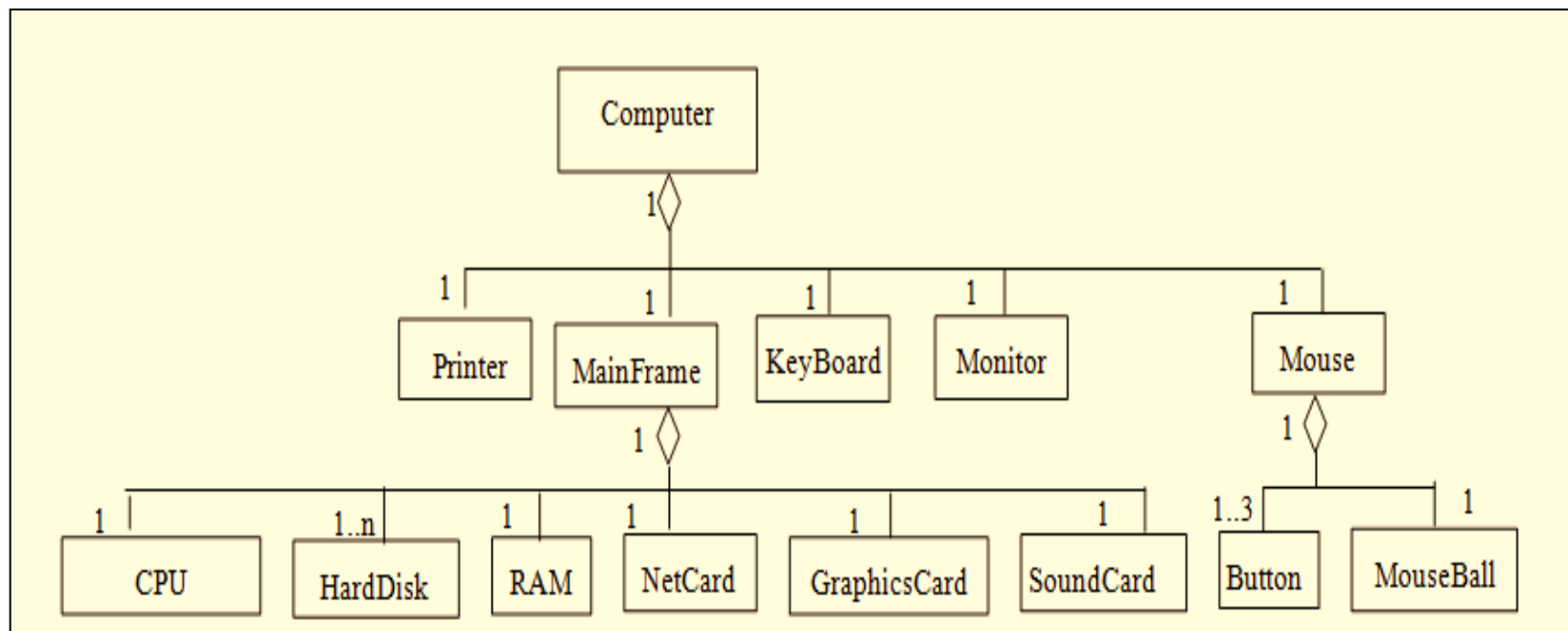
聚集关系 (Aggregation)

- 聚集指的是整体与部分之间的关系，在UML中用带实线的菱形箭头表示。
- 例如：台灯和灯泡之间就是聚集关系。

```
public class ReadingLamp{  
    private Bulb bulb;  
    private Circuit circuit;  
    ...  
}
```

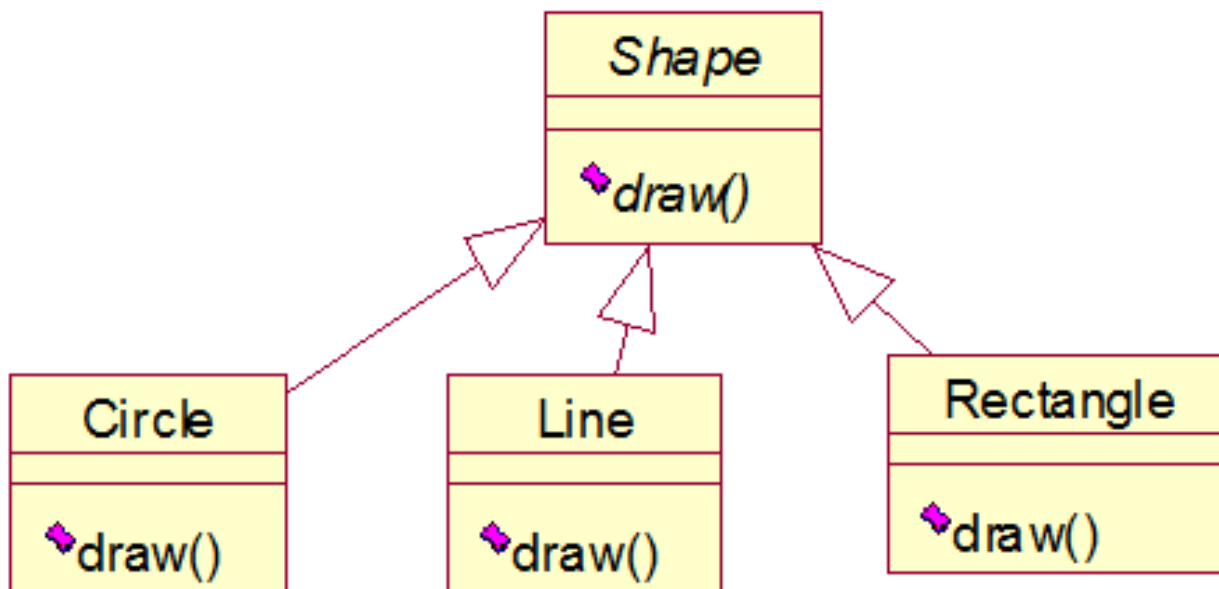


- 组合是一种用多个简单子系统来组装出复杂系统的有效手段。



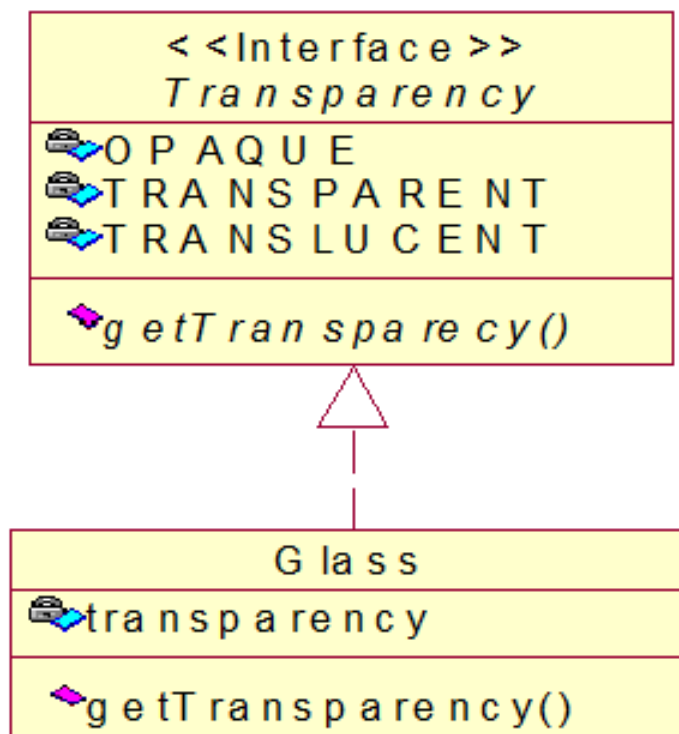
泛化关系 (Generalization)

- 泛化指的是类之间的继承关系



实现关系 (Realization)

- 实现指的是类与接口之间的关系。
- 在UML中用带虚线的三角形箭头表示，这里的接口指的是接口类型，接口名字用斜体字表示，接口中的方法都是抽象方法，也采用斜体字表示。





thank you!

高 祥

Email: xiang_gao@buaa.edu.cn

办公室：工训楼东603